~\OneDrive - Florida State University\FA25\Mech2\Project\Arduino\main\src\myDCEncoder\myDCEncoder.cpp

```cpp
# include "Arduino.h"
# include "myDCEncoder.h"

// This is a weird quirk of c++ and attachInterrupt
// Global static lookup for all encoder pins
myDCEncoder* encoder_lookup[NUM_DIGITAL_PINS] = {nullptr};

myDCEncoder::myDCEncoder(int IN_1, int IN_2, int EN, int ch_a, int ch_b)
{
    _IN_1 = IN_1;
    _IN_2 = IN_2;
    _EN = EN;

    _input_ch[0] = ch_a;
    _input_ch[1] = ch_b;

}


void myDCEncoder::init() {
    // Make IN1 and IN2 output pins
    pinMode(_IN_1, OUTPUT);
    pinMode(_IN_2, OUTPUT);

    // Make all the encoder pins inputs
    for (int i = 0; i < 2; i++) {
    pinMode(_input_ch[i], INPUT_PULLUP);
    }
}

void myDCEncoder::driveMotor(int PWM){

    // Deal with switching directions
    if (PWM > 0){
        digitalWrite(_IN_1, HIGH);
        digitalWrite(_IN_2, LOW);
    }
    else if (PWM <= 0){
        digitalWrite(_IN_1, LOW);
        digitalWrite(_IN_2, HIGH);
    }

    // Set the speed
    analogWrite(_EN, abs(PWM));
}
void myDCEncoder::PID(int desPos){
```

```
    // desPos will be determined as a function of compression when using the hall effect. It's useful to have a baseline PID function, though
    // REMEMBER CURPOS IS IN "TICKS"
    static  int last_p_error = 0;
    static unsigned int last_time = millis();
    static float i_error = 0; // integral of error
    static int stationaryCounter = 0;
    static int lastStationaryPosition = 0;

    // get timing info
    int now = millis();
    int dt = now - last_time;

    // find errors
    int p_error = desPos - _curPos;                         // P
    float d_error = (last_p_error - p_error) / dt;          // D
    i_error += -p_error * dt / 1000.;                       // I

    // reset for next time
    last_p_error = p_error;
    last_time = now;

    // Calculate commanded PWM (speed)
    float PwmCommand = _kp * p_error + _kd * d_error + _ki * i_error;

    if (PwmCommand > 255){
        PwmCommand = 255;
    }
    else if (PwmCommand < -255){
        PwmCommand = -255;
    }


    // increment if you've been stationary
    if (abs(p_error) <= 1){
        stationaryCounter ++;
    }
    else{
        stationaryCounter = 0;
    }

    // flush integral error if you've been stationary for a while
    if (stationaryCounter >= 15){
        i_error = 0;
        lastStationaryPosition = desPos;
        // in case desPos changes, reset all of this logic
        if (desPos != lastStationaryPosition){
            stationaryCounter = 0;
        }
    }
```

```
96
97
98
99      myDCEncoder::driveMotor(PwmCommand);
100
101 }
102
103 void myDCEncoder::ticksToDeg(){
104
105 }
106
107 //* INCLUDE CONVERSION TO DEGREES IN HERE!
108 // Function called by the ISRs to update the current position of the motor
109 void myDCEncoder::updateCurState(){
110     // an interrupt in the main script will call this. Based on the last channel that was
    interrupted, you'll know which direction you spun
111     static int lastState = 0;
112     int curState = 0;
113     static int lastTime = millis();
114
115     int curTime = millis();
116     int dt = millis() - lastTime;
117     lastTime = curTime;
118
119     bool a = digitalRead(_input_ch[0]);
120     bool b = digitalRead(_input_ch[1]);
121
122     if (!a&&!b){
123         curState = 0;
124     }
125     else if(a&&!b){
126         curState = 1;
127     }
128     else if(a&&b){
129         curState = 2;
130     }
131     else if(!a&&b){
132         curState = 3;
133     }
134
135     // cw is 0, 1, 2, 3
136     switch (curState){
137         case 0:
138         if (lastState == 3){
139             _curPos ++;
140             _curOmega = 1/dt;
141         }
142         else if (lastState == 1){
143             _curPos --;
144             _curOmega = -1/dt;
```

```
        }
        else if (lastState == 0){
            _curOmega = 0;
        }
        break;

        case 1:
        if (lastState == 0){
            _curPos ++;
            _curOmega = 1/dt;
        }
        else if (lastState == 2){
            _curPos --;
            _curOmega = -1/dt;
        }
        else if (lastState == 1){
            _curOmega = 0;
        }
        break;

        case 2:
        if (lastState == 1){
            _curPos ++;
            _curOmega = 1/dt;
        }
        else if (lastState == 3){
            _curPos --;
            _curOmega = -1/dt;
        }
        else if (lastState == 2){
            _curOmega = 0;
        }
        break;

        case 3:
        if (lastState == 2){
            _curPos ++;
            _curOmega = 1/dt;
        }
        else if (lastState == 0){
            _curPos --;
            _curOmega = -1/dt;
        }
        else if (lastState == 3){
            _curOmega = 0;
        }
        break;
    }

    lastState = curState;
```

```
}

```